

OBSTACLE AVOIDING ROBOT

Arduino UNO | HC-SR04 | L298N | SG90 Servo

1. Project Overview

The Obstacle Avoiding Robot is an autonomous ground vehicle capable of detecting and navigating around physical obstacles in its environment without any human intervention. Built on the Arduino UNO platform, the robot leverages an HC-SR04 ultrasonic distance sensor mounted on a servo motor to continuously scan its surroundings, and an L298N motor driver to independently control two DC geared wheels.

When an obstacle is detected within a threshold distance, the robot halts, performs a left-right scan using the servo, reverses, and steers clear — resuming forward motion once the path is clear. This project demonstrates the integration of sensing, actuation, and real-time decision-making in an embedded system.

2. Project Objectives

- Design and assemble an autonomous wheeled robot capable of real-time obstacle detection.
- Interface the HC-SR04 ultrasonic sensor with the Arduino UNO for accurate distance measurement.
- Use an SG90 servo motor to pan the sensor and scan for alternate paths.
- Drive two DC geared motors via the L298N H-bridge motor driver for controlled movement.
- Write and upload Arduino firmware implementing the obstacle avoidance logic.
- Verify system performance through physical testing on a flat surface.

3. Components and Hardware

The following table lists all hardware components used in the build, along with their specifications and functional roles within the system.

No.	Component	Model / Specification	Function / Role
1	Arduino UNO Board	ATmega328P Microcontroller	Main processing unit; runs the control logic
2	Motor Driver Board	L298N Dual H-Bridge	Drives both DC geared motors; controls speed and direction
3	Ultrasonic Sensor	HC-SR04 (2 cm – 400 cm)	Measures distance to obstacles by emitting and receiving ultrasonic pulses
4	Servo Motor	SG90 (0° – 180°)	Rotates the ultrasonic sensor to scan left and right for path selection
5	Geared DC Motors	With rubber wheels (x2)	Provide locomotion; controlled via the L298N driver
6	Battery Pack	4 x AA (6 V) with on/off switch	Powers the Arduino, motors, and all peripheral components
7	Jumper Wires	Male-to-Male / Male-to-Female	Interconnect all components on the robot chassis

4. Circuit Diagram

The wiring diagram below illustrates the complete electrical connections between the Arduino UNO, HC-SR04 ultrasonic sensor, SG90 servo motor, L298N motor driver board, DC geared motors, and 9V DC power supply.

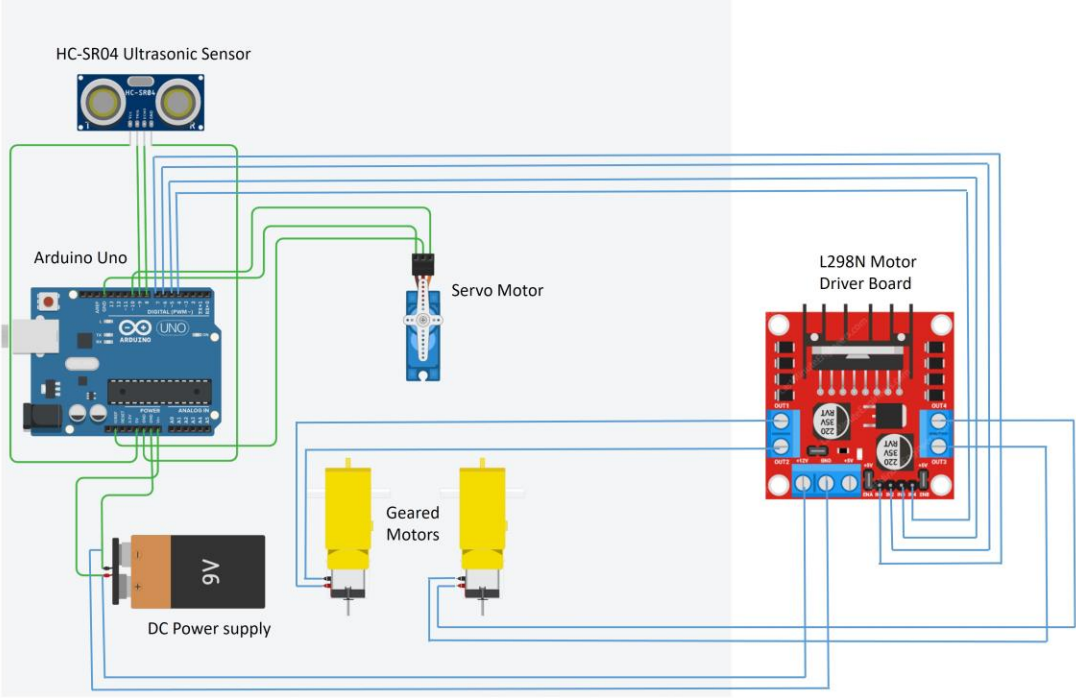


Figure 1: Complete Circuit Diagram — Obstacle Avoiding Robot

5. Pin Configuration

The table below maps each Arduino UNO digital pin to its connected peripheral and describes the signal type and functional purpose.

Arduino Pin	Connected To	Signal Type	Purpose
Pin 9 (TRIG)	HC-SR04 Trig	Digital Output	Triggers ultrasonic pulse
Pin 8 (ECHO)	HC-SR04 Echo	Digital Input	Receives reflected pulse
Pin 10 (PWM)	SG90 Servo Signal	PWM Output	Controls servo rotation angle
Pin 4 (MLa)	L298N IN1	Digital Output	Left motor direction control A
Pin 5 (MLb)	L298N IN2	Digital Output	Left motor direction control B
Pin 6 (MRa)	L298N IN3	Digital Output	Right motor direction control A
Pin 7 (MRb)	L298N IN4	Digital Output	Right motor direction control B

Arduino Pin	Connected To	Signal Type	Purpose
5V / GND	HC-SR04, Servo, L298N	Power	Component power supply

6. Software and Firmware

6.1 Development Environment

- Platform: Arduino IDE
- Language: C++ (Arduino framework)
- Library: Servo.h (bundled with Arduino IDE)
- Serial Baud Rate: 9600 bps (used for distance and angle debugging via Serial Monitor)

6.2 Source Code

The complete firmware is listed below. The code is structured into modular helper functions for distance measurement, motor control, and a main loop() that continuously sweeps the servo and evaluates obstacle presence.

```
#include <Servo.h>
Servo Myservo;
#define trigPin 9
#define echoPin 8
#define MLa 4
#define MLb 5
#define MRa 6
#define MRb 7
int servoAngle = 90;
int servoStep = 5;
long lastDist = 999;

long getDistance() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long dur = pulseIn(echoPin, HIGH, 20000);
    if (dur == 0) return 999;
    return dur / 58.2;
}

void moveForward() {
    digitalWrite(MRb, HIGH); digitalWrite(MRa, LOW);
    digitalWrite(MLb, HIGH); digitalWrite(MLa, LOW);
}

void moveBackward() {
    digitalWrite(MRb, LOW); digitalWrite(MRa, HIGH);
    digitalWrite(MLb, LOW); digitalWrite(MLa, HIGH);
}
```

```
void turnLeft() {
    digitalWrite(MRb, HIGH); digitalWrite(MRa, LOW);
    digitalWrite(MLb, LOW); digitalWrite(MLa, LOW);
}

void turnRight() {
    digitalWrite(MLb, HIGH); digitalWrite(MLa, LOW);
    digitalWrite(MRb, LOW); digitalWrite(MRa, LOW);
}

void stopMotors() {
    digitalWrite(MRb, LOW); digitalWrite(MRa, LOW);
    digitalWrite(MLb, LOW); digitalWrite(MLa, LOW);
}

void setup() {
    Serial.begin(9600);
    pinMode(MLa, OUTPUT);
    pinMode(MLb, OUTPUT);
    pinMode(MRa, OUTPUT);
    pinMode(MRb, OUTPUT);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    Myservo.attach(10);
    Myservo.write(90);
    delay(1000);
}

void loop() {
    servoAngle += servoStep;
    if (servoAngle >= 160 || servoAngle <= 20) {
        servoStep = -servoStep;
    }
    Myservo.write(servoAngle);
    delay(30);
    long dist = getDistance();
    Serial.print("Angle: "); Serial.print(servoAngle);
    Serial.print("   Dist: "); Serial.println(dist);

    if (servoAngle > 60 && servoAngle < 120 && dist < 20) {
        stopMotors();
        delay(300);
        Myservo.write(0); delay(600);
        long rightDist = getDistance();
        Serial.print("Right: "); Serial.println(rightDist);
        Myservo.write(180); delay(600);
        long leftDist = getDistance();
        Serial.print("Left: "); Serial.println(leftDist);
        Myservo.write(90); delay(400);
        moveBackward();
        delay(400);
        stopMotors();
        delay(200);
        if (leftDist >= rightDist) {
            turnLeft();
        } else {
            turnRight();
        }
        delay(450);
        stopMotors();
        delay(200);
    } else {
        moveForward();
    }
}
```

7. Working Principle

7.1 Distance Measurement

The HC-SR04 sensor measures distance by emitting a short 10-microsecond ultrasonic burst on the TRIG pin. The echo pin then goes HIGH for a duration proportional to the round-trip travel time of the reflected sound wave. The firmware calculates distance in centimeters using the formula:

$$\text{Distance (cm)} = \text{Pulse Duration } (\mu\text{s}) \div 58.2$$

A timeout of 20,000 μs is applied to `pulseIn()` to prevent the loop from hanging when no echo is received. In such cases, the function returns 999, which is treated as a clear path.

7.2 Decision Logic

The servo continuously sweeps between 20° and 160° in steps of 5° while the robot drives forward, functioning like a radar. Distance is read at each step. Obstacle avoidance is only triggered when the servo is facing roughly forward (between 60° and 120°) and the measured distance is less than 20 cm. This prevents false triggers from side walls or nearby objects outside the robot's path.

When an obstacle is confirmed ahead, the robot executes the following avoidance sequence.

7.3 Avoidance Manoeuvre Sequence

Step 1 — Stop: All motor outputs are set LOW. The robot halts immediately upon obstacle detection.

Step 2 — Deliberate Scan: The servo performs a focused scan — turning to 0° (right) and reading distance, then turning to 180° (left) and reading distance. Both values are stored for direction comparison. The servo returns to 90° (centre) after scanning.

Step 3 — Reverse: Both motors spin in reverse for 400 ms to create clearance from the obstacle.

Step 4 — Smart Turn: The robot turns toward whichever side recorded the greater distance during the scan. If left distance is greater or equal, it turns left (right motor forward, left motor stopped). If right distance is greater, it turns right (left motor forward, right motor stopped). Turning continues for 450 ms.

Step 5 — Resume Forward: Motors stop briefly, then the main loop resumes — servo sweeping and forward motion continue as normal.

8. Results and Observations

The robot was tested on a flat indoor surface with rigid obstacles placed at varying distances. Key observations are summarised below:

- The HC-SR04 sensor provided reliable measurements in the range of 3 cm to 200 cm under standard indoor lighting conditions.
- The servo sweep radar approach successfully detected obstacles before collision in the majority of test runs.
- The smart turn logic — choosing direction based on left vs right scan distance — resulted in more effective avoidance compared to fixed-direction turning.

- Occasional false triggers were noted when the robot operated on highly reflective or angled surfaces, slightly scattering the ultrasonic beam.
- Serial Monitor output confirmed real-time angle and distance readings at 9600 bps, which proved useful during threshold tuning.

9. Limitations and Future Improvements

9.1 Known Limitations

- Single sensor: Only one ultrasonic sensor is used; narrow or multiple simultaneous obstacles may not be detected correctly.
- Fixed turn duration: The turn step of 450 ms may under-rotate or over-rotate depending on surface friction and battery charge level.
- Servo sweep blind spot: When the servo is near 20° or 160° (extreme ends of sweep), forward obstacles may briefly go undetected.
- 2D sensing only: The sensor is mounted at a fixed height and cannot detect low-lying or overhead obstacles.

9.2 Suggested Improvements

- Add a second ultrasonic sensor or IR sensors on the sides for wider environmental awareness.
- Implement turn-until-clear logic — keep turning and checking front distance after each chunk rather than turning for a fixed duration.
- PID speed control using the L298N ENA/ENB PWM pins for smoother and more energy-efficient motion.
- Integrate a Bluetooth module (e.g., HC-05) for remote override and telemetry logging.
- Add an OLED display to show real-time sensor angle, distance, and robot state on the chassis.

10. Conclusion

The Obstacle Avoiding Robot successfully demonstrates the practical integration of ultrasonic sensing, servo-based radar scanning, and intelligent differential motor drive within the Arduino ecosystem. The updated firmware introduces continuous servo sweeping during forward motion and direction-aware avoidance logic, resulting in more reliable and adaptive obstacle navigation compared to fixed-direction approaches.

The system performed reliably under standard test conditions, and the modular code structure allows for straightforward future upgrades. With the enhancements described in Section 9, the platform can evolve into a capable autonomous navigation testbed.